

```

1  n_rows = input("Enter n: ");
2  n_cols = input("Enter m: ");
3
4  A = input("Enter the matrix A: ");
5
6  supply = input("Enter the supply vector: ");
7  demand = input("Enter the demand vector: ");
8
9  %%
10 allocation = zeros(n_rows, n_cols);
11
12 s = supply(:);
13 d = demand(:);
14 c = A;
15
16 while any(s > 0) && any(d > 0)
17     row_penalties = zeros(n_rows, 1);
18     for i = 1:n_rows
19         sorted_row = sort(c(i, :));
20         row_penalties(i) = sorted_row(2) - sorted_row(1);
21     end
22
23     row_penalties;
24
25     col_penalties = zeros(n_cols, 1);
26     for j = 1:n_cols
27         sorted_col = sort(c(:, j));
28         col_penalties(j) = sorted_col(2) - sorted_col(1);
29     end
30
31     col_penalties;
32
33     [max_row_p, r_idx] = max(row_penalties);
34     [max_col_p, c_idx] = max(col_penalties);
35
36     if max_row_p >= max_col_p
37         selected_row = r_idx;
38         [~, selected_col] = min(c(selected_row, :));
39     else
40         selected_col = c_idx;
41         [~, selected_row] = min(c(:, selected_col));
42     end
43
44     alloc_amt = min(s(selected_row), d(selected_col));
45     allocation(selected_row, selected_col) = alloc_amt;
46
47     s(selected_row) = s(selected_row) - alloc_amt;
48     d(selected_col) = d(selected_col) - alloc_amt;
49
50     if s(selected_row) == 0
51         c(selected_row, :) = Inf;
52     end
53     if d(selected_col) == 0
54         c(:, selected_col) = Inf;
55     end
56 end
57
58

```

```

59 allocation
60
61 total_cost = sum(sum(allocation .* A))
62
63
64 %%
65
66 while(1)
67     u = zeros(n_rows, 1);
68     v = zeros(1, n_cols);
69
70     col_flag = zeros(1, n_cols);
71     row_flag = zeros(n_rows, 1);
72
73     row_flag(1) = 1;
74     for j=1:m
75         if allocation(1,j) > 0
76             v(j) = A(1,j);
77             [u, v, col_flag, row_flag] = handle_col(A, allocation, j, u, v, col_flag,
row_flag, n_rows, n_cols);
78         end
79     end
80
81     u
82     v
83
84
85     constraint_check = zeros(n_rows, n_cols);
86     for i=1:n_rows
87         for j=1:n_cols
88             constraint_check(i,j) = u(i) + v(j) - A(i,j);
89         end
90     end
91
92     constraint_check
93
94     max_val = -inf;
95     idx_r = -1;
96     idx_c = -1;
97     for i=1:n_rows
98         for j=1:n_cols
99             if constraint_check(i,j) > max_val
100                 max_val = constraint_check(i,j);
101                 idx_r = i;
102                 idx_c = j;
103             end
104         end
105     end
106
107     if max_val <= 0
108         total_cost = sum(sum(allocation .* A))
109
110         break
111
112     end
113
114
115     binary_mtx = zeros(n_rows, n_cols)

```

```

116     coords = [0 0 0];
117     k = 0;
118
119     for i=1:n_rows
120         for j=1:n_cols
121             if allocation(i,j) > 0
122                 binary_mtx(i,j) = 1;
123                 coords = [coords; i, j, 1];
124                 k = k + 1;
125             end
126         end
127     end
128
129     binary_mtx(idx_r,idx_c) = 1
130     coords = [coords; idx_r, idx_c, 1];
131     k = k+1;
132
133     coords = coords(2:end,:)
134
135     n = n_rows;
136     m = n_cols;
137
138     gridd = binary_mtx
139
140     loop_coords = [0 0];
141
142     for i=1:k
143         ai = coords(i,1);
144         aj = coords(i,2);
145
146         if coords(i,3) == -1
147             continue
148         end
149
150         % coords(i,3) = 2;
151
152         grid_cpy = gridd;
153         [flag, loop_coords] = find_loop_col(ai,aj,k, grid_cpy, coords, ai, aj,
loop_coords);
154         if ~flag
155             coords(i,3) = -1;
156         else
157             loop_coords = [loop_coords; ai, aj];
158             % disp([ai, aj]) %%%%%%%%%%%%%%%
159             break;
160         end
161     end
162
163     loop_coords = loop_coords(2:end,:)
164
165     ll = size(loop_coords, 1);
166     for i=1:ll
167         if loop_coords(i,:) == [idx_r, idx_c]
168             loop_coords = [loop_coords(i:end,:); loop_coords(1:i-1,:)];
169             break
170         end
171     end
172

```

```

173     loop_coords
174
175
176     min_loop = inf;
177     for i=2:2:ll
178         min_loop = min(min_loop, allocation(loop_coords(i,1), loop_coords(i,2)));
179     end
180
181     flag = 1
182     for i=1:ll
183         if flag == 1
184             allocation(loop_coords(i,1), loop_coords(i,2)) =
allocation(loop_coords(i,1), loop_coords(i,2)) + min_loop;
185             flag = -1;
186         else
187             allocation(loop_coords(i,1), loop_coords(i,2)) =
allocation(loop_coords(i,1), loop_coords(i,2)) - min_loop;
188             flag = 1;
189         end
190     end
191
192     total_cost = sum(sum(allocation .* A))
193
194 end
195
196 % disp("No Loop was found!")
197
198
199
200 %%
201
202 %%
203
204 function [found_bool, loop_coords] = find_loop_col(r,c,k, grid, coords, ai, aj,
loop_coords)
205     % disp([r,c])
206     found_bool = false;
207     in_this_col = false;
208     for coord=1:k
209         if coords(coord, 2) == c && coords(coord, 3) == 1
210             coords(coord, 3) = 2;
211             in_this_col = true;
212             r = coords(coord, 1);
213             [flag , loop_coords] = find_loop_row(r,c,k, grid, coords, ai, aj,
loop_coords);
214             if flag
215                 found_bool = true; %%%%%%%%%%%%%%%
216                 loop_coords = [loop_coords; r,c];
217                 % disp([r,c])
218                 return
219             end
220             coords(coord, 3) = 1;
221         end
222     end
223     if ~in_this_col
224         return
225     end
226

```

```

227 end
228
229 function [found_bool, loop_coords] = find_loop_row(r,c,k, grid, coords, ai, aj,
loop_coords)
230     % disp([r,c])
231     found_bool = false;
232     in_this_row = false;
233     for coord=1:k
234         if coords(coord, 1) == r && coords(coord, 3) == 1
235             coords(coord, 3) = 2;
236             in_this_row = true;
237             c = coords(coord, 2);
238             if r == ai && c == aj
239                 disp("Loop found!")
240                 found_bool = true;
241                 return
242             end
243             [flag, loop_coords] = find_loop_col(r,c,k, grid, coords, ai, aj,
loop_coords);
244             if flag
245                 found_bool = true;
246                 loop_coords = [loop_coords; [r,c]];
247                 % disp([r,c]) %%%%%%%%%%%%%%%
248                 return
249             end
250             coords(coord, 3) = 1;
251         end
252     end
253     if ~in_this_row
254         return
255     end
256
257 end
258
259 %%
260
261
262 function [u,v, col_flag, row_flag] = handle_col(A, allocation, j, u, v, col_flag,
row_flag, n_rows, n_cols)
263     col_flag(j) = 1;
264     for i=1:n_rows
265         if row_flag(i) == 0 && allocation(i,j) > 0
266             u(i) = A(i,j) - v(j);
267             [u, v, col_flag, row_flag] = handle_row(A, allocation, i, u, v, col_flag,
row_flag, n_rows, n_cols);
268         end
269     end
270 end
271
272 function [u,v, col_flag, row_flag] = handle_row(A, allocation, i, u, v, col_flag,
row_flag, n_rows, n_cols)
273     row_flag(i) = 1;
274     for j=1:n_cols
275         if col_flag(j) == 0 && allocation(i,j) > 0
276             v(j) = A(i,j) - u(i);
277             [u, v, col_flag, row_flag] = handle_col(A, allocation, j, u, v, col_flag,
row_flag, n_rows, n_cols);
278         end

```

279 end
280 end